AMBATI SHIVA SHANKAR REDDY

192425174

COURSE CODE – CSA0613

Question 1

Problem Statement

A university maintains student records where marks are almost sorted, except for a few recently updated entries. The system needs to sort the data frequently with minimal computation time. The developer is choosing between Selection Sort, Bubble Sort, and Insertion Sort.

a) Analyze all three algorithms in this context.

b) Recommend the most suitable one with justification based on comparisons and swaps.

Source Code (Python)

PythoN

```
# Question 1
# Comparison of Selection Sort, Bubble Sort and Insertion Sort


def selection_sort(arr):
    a = arr.copy()
    comparisons = 0
    swaps = 0
    n = len(a)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[min_index]:
                min_index = j
        if min_index != i:
            a[i], a[min_index] = a[min_index], a[i]
```

```
            swaps += 1

    return a, comparisons, swaps


def bubble_sort(arr):
    a = arr.copy()
    comparisons = 0
    swaps = 0
    n = len(a)

    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                swapped = True
        if not swapped:
            break

    return a, comparisons, swaps
def insertion_sort(arr):
    a = arr.copy()
    comparisons = 0
    swaps = 0

    for i in range(1, len(a)):
        key = a[i]
```

```
        j = i - 1

        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                swaps += 1
                j -= 1
            else:
                break

        a[j + 1] = key

    return a, comparisons, swaps

# Sample student marks (Almost Sorted)
marks = [45, 52, 58, 61, 67, 72, 70, 78, 83, 91]

print("Original Marks:")
print(marks)

print("\nSelection Sort")
sorted_marks, comp, sw = selection_sort(marks)
print("Sorted:", sorted_marks)
print("Comparisons:", comp)
print("Swaps:", sw)

print("\nBubble Sort")
sorted_marks, comp, sw = bubble_sort(marks)
print("Sorted:", sorted_marks)
```

print("Comparisons:", comp)

print("Swaps:", sw)

print("\nInsertion Sort")

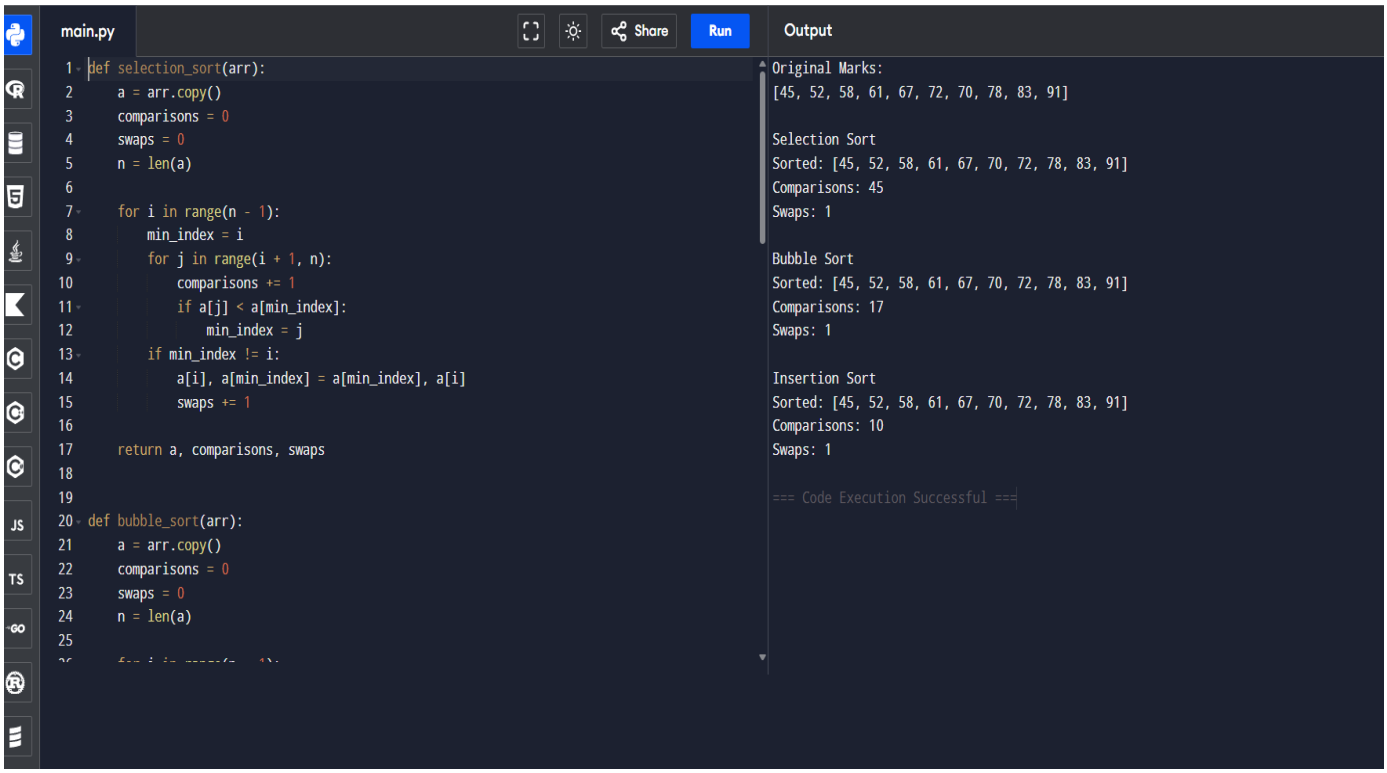
sorted_marks, comp, sw = insertion_sort(marks)

print("Sorted:", sorted_marks)

print("Comparisons:", comp)

print("Swaps:", sw)

OUTPUT



Sample Output

Original Marks:

[45, 52, 58, 61, 67, 72, 70, 78, 83, 91]

Selection Sort

Sorted: [45, 52, 58, 61, 67, 70, 72, 78, 83, 91]

Comparisons: 45

Swaps: 1

Bubble Sort

Sorted: [45, 52, 58, 61, 67, 70, 72, 78, 83, 91]

Comparisons: 17

Swaps: 1

Insertion Sort

Sorted: [45, 52, 58, 61, 67, 70, 72, 78, 83, 91]

Analysis

Selection Sort

Working

- Finds the minimum element.
- Places it in the correct position.
- Repeats for all elements.

Time Complexity

- Best Case : $O(n^2)$
- Average Case : $O(n^2)$
- Worst Case : $O(n^2)$

Advantages

- Performs fewer swaps.
- Easy to implement.

Disadvantages

- Always compares every element.
- Does not benefit from nearly sorted data.

Bubble Sort

Working

- Compares adjacent elements.
- Swaps them if they are in the wrong order.

- Stops early if no swaps occur.

Time Complexity

- Best Case : O(n) (with optimization)
- Average Case : $O(n^2)$
- Worst Case : $O(n^2)$

Advantages

- Detects already sorted arrays.
- Simple implementation.

Disadvantages

- Performs many unnecessary swaps.
- Less efficient than Insertion Sort for nearly sorted arrays.

Insertion Sort

Working

- Takes one element at a time.
- Inserts it into the correct position in the sorted portion.

Time Complexity

- Best Case : O(n)
- Average Case : $O(n^2)$
- Worst Case : $O(n^2)$

Advantages

- Very efficient for nearly sorted data.
- Requires fewer comparisons.
- Requires fewer element movements.
- Stable sorting algorithm.

Disadvantages

- Not suitable for very large random datasets.

Comparison Table

Feature	Selection Sort	Bubble Sort	Insertion Sort
Best Case	$O(n^2)$	O(n)	O(n)
Average Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Stable	No	Yes	Yes
Adaptive	No	Yes	Yes
Swaps	Few	Many	Few
Suitable for Nearly Sorted Data	No	Good	Excellent

Recommendation.

Insertion Sort is the most suitable algorithm because:

- It performs very few comparisons for nearly sorted data.
- It requires minimal element movements.
- It has a best-case time complexity of O(n).
- It is adaptive, meaning it efficiently handles data that is already mostly sorted.
- It maintains the relative order of equal elements (stable sorting).

Although Selection Sort performs fewer swaps, it still performs $O(n^2)$ comparisons regardless of the input order. Bubble Sort improves with an early-exit optimization but generally performs more swaps than Insertion Sort.

Therefore, Insertion Sort is recommended for frequently sorting almost sorted student records because it provides the best practical performance with minimal computation.

Conclusion

Among the three sorting algorithms, Insertion Sort is the best choice for maintaining university student records that are nearly sorted. It minimizes comparisons and element movements, making it faster and more efficient than Selection Sort and Bubble Sort in this scenario.